



Cultura
Secretaría de Cultura



ORIGINAL



unesco



PLATAFORMA
ATLAS NACIONAL DE
TÉCNICAS DEL ARTE TEXTIL

MANUAL DE ADMINISTRADOR

Contenido

Contenido	2
1. Vision general del sistema	3
1.1 Donde esta alojada la plataforma.....	3
2. Estructura de archivos	3
3. Arquitectura del codigo.....	4
3.1 Flujo de carga	4
3.2 Modelo de datos (objeto ATLAS)	4
3.3 Modulos funcionales de main.js.....	5
3.4 Librerias externas	5
4. Estructura de los archivos CSV	6
4.1 data_by_technique_id.csv.....	6
4.2 data_by_record_id.csv.....	7
4.3 indice_imagenes.csv	7
5. Como actualizar el atlas	8
5.1 Actualizar datos de tecnicas	8
5.2 Actualizar la clasificacion experta	8
5.3 Agregar o actualizar imagenes.....	8
5.4 Modificar textos del reporte	9
6. Datos: arquitectura actual y migracion a API.....	9
6.1 Arquitectura actual: CSV estaticos.....	9
6.2 Como migrar a una API en el futuro	9
6.3 Consideraciones para una API	10
7. Mantenimiento y diagnostico	10
7.1 Verificar que la plataforma carga correctamente	10
7.2 El mapa no muestra los estados de Mexico	11
7.3 Actualizacion de librerias.....	11
8. Formularios de contribucion (KoboToolbox)	11
9. Sistema de diseno.....	11

1. Visión general del sistema

El Atlas Nacional de Técnicas del Arte Textil es una aplicación web estática compuesta exclusivamente de archivos HTML, CSS, JavaScript y CSV. No tiene servidor propio, no usa base de datos, y no requiere ningún lenguaje de programación del lado del servidor (como PHP o Python en producción).

Todo el procesamiento de datos ocurre en el navegador del usuario: al cargar la página, el navegador descarga los archivos CSV y construye la interfaz completa en memoria. Esto hace que el sistema sea extremadamente fácil de alojar y de mantener.

Principio de diseño fundamental: ningún dato está embebido en el código HTML ni en el JavaScript. Toda la información se lee en tiempo de ejecución desde los archivos CSV. Para actualizar el contenido del atlas basta con reemplazar los CSV y recargar la página.

1.1 Donde está alojada la plataforma

La plataforma está alojada en los servidores de la Oficina de UNESCO en México. La URL de acceso pública es: <https://www.unesco.org/mexico/atlas-textil>

Para publicar actualizaciones, el administrador debe subir los archivos modificados al servidor de UNESCO México siguiendo el proceso de publicación que define el equipo de tecnología de esa oficina.

2. Estructura de archivos

El proyecto completo se encuentra en una sola carpeta. Todos los archivos deben estar en el mismo directorio para que las referencias relativas entre ellos funcionen correctamente.

```
atlas/  
  index.html      <- Plataforma principal  
  main.js         <- Lógica completa de la aplicación  
  styles.css      <- Estilos globales  
  atlas_textil_v2.html <- Reporte "Acerca del Atlas"  
  data_by_technique_id.csv <- Datos por técnica (fuente canónica)  
  data_by_record_id.csv <- Datos por registro individual  
  indice_imagenes.csv <- Índice de imágenes  
  imagenes/       <- Carpeta de fotos (si las hay localmente)
```

Los archivos CSV deben estar siempre en la misma carpeta que index.html. Si se mueven o renombran, la plataforma mostrará un error de carga.

Archivo	Función
index.html	Punto de entrada. Contiene la estructura HTML de las seis vistas, el encabezado y los modales. Referencia main.js y styles.css.
main.js	Toda la lógica de la aplicación: carga de CSV, construcción del modelo de datos, renderizado de cada vista, interacciones. Es el archivo central del sistema.
styles.css	Diseño visual completo: paleta de colores, tipografía (Francés, DM Sans, Pase Mono), layouts, animaciones, componentes.

atlas_textil_v2.html	Reporte standalone de metodología y estadísticas. Se abre como overlay. Carga sus propios CSV de forma independiente.
data_by_technique_id.csv	Fuente canónica de datos estructurados por técnica. Una fila = una técnica.
data_by_record_id.csv	Datos cualitativos por registro de campo. Una fila = una ficha levantada.
indice_imagenes.csv	Mapeo entre archivos de imagen y técnicas. Una fila = un archivo de imagen.

3. Arquitectura del código

3.1 Flujo de carga

El ciclo de vida completo de la aplicación al abrirse en un navegador es:

1. El navegador descarga index.html y muestra el overlay de carga.
2. index.html carga main.js y styles.css en paralelo.
3. main.js ejecuta loadCSVs(): descarga los tres CSV en paralelo con fetch().
4. Los CSV se parsean con el parser propio (sin dependencias externas).
5. buildAtlasFromCSV() construye el modelo de datos unificado ATLAS en memoria.
6. Se actualizan las estadísticas del encabezado (técnicas, registros, estados).
7. Se inicializa la vista activa (mapa por defecto) y se oculta el overlay.
8. Las demás vistas se inicializan bajo demanda cuando el usuario navega a ellas.

3.2 Modelo de datos (objeto ATLAS)

El objeto global ATLAS es la fuente de verdad de toda la aplicación una vez cargada. Tiene la siguiente estructura:

```
ATLAS = {
  tecnicas: [ // Array de objetos, uno por tecnica
    {
      tecnica: string,      // Nombre canonico
      cat1, cat2, cat3, cat4, // Clasificacion experta
      grupo: string,       // Categoría CAT-N-1
      estados: string[],   // Estados donde se practica
      lenguas: string[],   // Lenguas indígenas asociadas
      municipios: string[], // Municipios (de record_id)
      n_fichas: number,    // Total de registros
      n_mujeres, n_hombres, // Desglose de genero
      edad_promedio: number,
      manufactura: { mano, pedal, telar, ... },
      tenido: { plantas, minerales, animales, otro },
      aprendizaje: { madre, abuela, tia, ... },
      ensenanza: { hijas, nietos, estudiantes, ... },
      materiales: string,
      historia: string,    // Texto mas extenso de record_id
      significados: string[], // Fragmentos S1-S5
      imagenes: string[],  // Nombres de archivo
      temporalidad: "antigua"|"nueva",
    }
  ]
}
```

```

],
estados: { // Mapa estado -> lista de nombres de tecnicas
  "Oaxaca": ["Telar de cintura", "Brocado", ...],
  ...
}
}
}

```

3.3 Módulos funcionales de main.js

main.js está organizado en módulos funcionales secuenciales. Cada sección está delimitada por comentarios de separación.

Modulo / función principal	Responsabilidad
parseCSV / parseCSVRow	Parser CSV propio sin dependencias. Maneja campos entre comillas, comas dentro de valores y saltos de linea.
buildAtlasFromCSV	Construye el objeto ATLAS cruzando los tres CSV. Contiene la lógica de que campo viene de cual fuente.
loadCSVs	Carga asíncrona en paralelo de los tres CSV. Maneja errores de red y muestra el banner de error.
initMap / stateStyle / onStateClick	Inicializa el mapa Leaflet, aplica estilos coroplético, maneja clics y hover de estados.
initMapFilter / applyMapFilter	Panel de filtros jerárquico. Construye los chips en cascada y actualiza los estilos del mapa según el filtro.
renderCatalog	Renderiza la cuadrícula de tarjetas del catálogo con búsqueda y filtros por categoría.
openFicha / closeFicha	Modal de ficha detalle. Construye dinámicamente el HTML de cada sección de la ficha.
initNetwork / drawNetwork	Grafo de fuerza sobre canvas. Implementación propia sin dependencias (sin D3). Maneja zoom, arrastre y resaltado.
initArbol	Árbol jerárquico con expansión progresiva. Construye el árbol desde las columnas CAT-N-* del CSV.
initTenidoView / renderTenidoMatrix	Matriz de técnicas x tipos de tenido con resaltado por filtro.
initReportCharts	Graficas del reporte "Acerca del Atlas" usando Chart.js.

3.4 Librerías externas

La plataforma usa tres librerías externas, todas cargadas desde CDN públicos:

Librería	Versión y uso
Leaflet.js	1.9.4 — Mapa interactivo y capas GeoJSON de estados de México.
Chart.js	4.4.0 — Graficas del reporte "Acerca del Atlas" (barras, donas).
Google Fonts	Sin versión fija — Tipografías Fraunces, DM Sans y Space Mono.

El grafo de la Red de Técnicas es una implementación propia sobre canvas, sin D3 ni Vis.js. Esto reduce las dependencias y el peso de carga de la página.

4. Estructura de los archivos CSV

Los tres archivos CSV son el corazón del sistema. La plataforma se actualiza completamente reemplazando estos archivos. A continuación, se documenta cada columna y su uso.

4.1 data_by_technique_id.csv

Una fila por técnica. Es la fuente canónica de todos los datos estructurados y cuantificados. Este archivo es generado por el proceso de procesamiento de datos del proyecto.

Columna	Descripción
Tecnica	Nombre canónico estandarizado de la técnica. Es la clave primaria: debe coincidir con el campo Técnica de los otros dos CSV.
tecnica_norm	Nombre normalizado alternativo. Se usa si Técnica este vacío.
n_fichas	Total de registros individuales que documentan esta técnica.
n_mujeres / n_hombres	Desglose de registros por genero de artesano.
edad_prom	Edad promedio de los artesanos que practican esta técnica.
estados	Lista de estados separados por coma donde se registró la técnica.
n_man_mano / n_man_telar / n_man_pedal / ...	Numero de registros de cada tipo de manufactura.
n_tenido plantas / n_tenido minerales / n_tenido animales / n_tenido otro	Numero de registros de cada tipo de tenido.
n_aprendio_madre / _abuela / _tia / _hermana / _cunada / _instructor / _padre	Con cuantos registros el artesano aprendió de cada figura.
n_ens_hijas / _nietos / _sobrinos / _pareja / _estudiantes / _hijos	A quienes enseñan la técnica.
n_no_ha_enseñado	Cuantos artesanos no han enseñado la técnica aún.
ceremonia_principal / n_ceremonia_principal / ceremonias_resumen	Ceremonias más comunes y conteos.
prenda_principal / n_prenda_principal / prendas_resumen	Prendas más comunes y conteos.
Materiales_concat_clean	Cadena de materiales usados, depurada y concatenada.
CAT-N-1	Categoría experta de nivel 1: Hilados, Tenidos, Tejido, Técnicas decorativas, Acabados.
CAT-N-2	Subcategoría de nivel 2 (por ejemplo: Bordados, Tejidos, Anudados).
CAT-N-3	Tipo de nivel 3.
CAT-N-4	Variante de nivel 4 (no todas las técnicas llegan a este nivel).

uuid	Identificador único interno del registro.
count	Campo de conteo auxiliar del procesamiento.

4.2 data_by_record_id.csv

Una fila por ficha individual levantada en campo. Este archivo proviene directamente de KoboToolbox con mínima transformación. Se usa para campos cualitativos y para calcular listas únicas (lenguas, municipios) que no están disponibles en data_by_technique_id.

Columna clave	Descripción
Tecnica	Nombre de la técnica. Clave de unión con data_by_technique_id.
Sede	Sede donde se levantó la ficha: CDMX, Mérida o Tijuana.
Genero	Genero declarado por el artesano: mujer, hombre, no especificar.
Edad	Edad del artesano al momento del registro.
Estado / Municipio / Localidad	Ubicación geográfica del artesano.
Lengua	Lengua indígena que habla el artesano.
Historia	Texto libre sobre la historia de la técnica. Se usa el fragmento más extenso por técnica en la ficha.
S1 a S5	Fragmentos de significado cultural. Se usan los más extensos como testimonios en la ficha.
Temporalidad	Si la técnica se considera antigua o nueva. Se calcula por mayoría de registros por técnica.
Madre / Abuela / Tia / Hermana / Cunada / Instructora / Padre	Valores binarios (1/0): de quien aprendió el artesano. Usados en las gráficas del reporte.
Hijas / Nietos / Sobrinos / ...	Valores binarios: a quien enseñan.
pic / video	Rutas o identificadores de archivos multimedia asociados.
uuid / _index	Identificadores únicos del registro en KoboToolbox.

Las columnas pr1-pr10, c1-c8, cm1-cm8 son campos internos de KoboToolbox (números de pregunta y respuesta codificada). No son usadas directamente por la plataforma.

4.3 indice_imagenes.csv

Una fila por archivo de imagen disponible. Mapea cada imagen a la técnica que representa.

Columna	Descripción
archivo_descargado	Nombre del archivo de imagen tal como está almacenado en el servidor. Es la clave que usa la plataforma para construir la URL de la imagen.
Tecnica	Nombre de la técnica a la que corresponde la imagen. Debe coincidir con el campo Técnica de data_by_technique_id.csv.
status	Estado de la descarga del archivo: "ok" o "error". Solo se incluyen imágenes con status "ok".
Sede / Estado / Municipio	Metadatos geográficos del registro de donde proviene la imagen.

Genero / tecnica_grupo	Metadatos adicionales del registro.
url / id / error_msg	URL original de KoboToolbox, ID interno y mensaje de error si aplica.

5. Como actualizar el atlas

5.1 Actualizar datos de técnicas

Para agregar, modificar o eliminar técnicas del atlas:

9. Generar los nuevos archivos `data_by_technique_id.csv` y/o `data_by_record_id.csv` con el proceso de procesamiento de datos del proyecto (Python/Pandas).
10. Verificar que los nombres en la columna "Técnica" sean consistentes entre ambos archivos y con `indice_imagenes.csv`.
11. Subir los nuevos CSV al servidor de UNESCO México, reemplazando los archivos anteriores en la misma ruta.
12. Recargar la plataforma en el navegador (F5). No es necesario modificar ningún archivo HTML ni JS.

IMPORTANTE: No cambiar los nombres de los archivos CSV. La plataforma los referencia con nombres exactos: `data_by_technique_id.csv`, `data_by_record_id.csv` e `indice_imagenes.csv`.

5.2 Actualizar la clasificación experta

Las columnas CAT-N-1, CAT-N-2, CAT-N-3 y CAT-N-4 de `data_by_technique_id.csv` son las que determinan la clasificación mostrada en el árbol, los chips de filtro del mapa y el catálogo. Para modificar la clasificación de una técnica basta con editar esas columnas en el CSV y subir el archivo actualizado.

Los colores de cada categoría están definidos en el objeto `CAT1_COLOR` al inicio de `main.js`. Si se agrega una nueva categoría de nivel 1 que no esté en esa tabla, se le asigna el color neutro (arena). Para asignarle un color específico hay que agregar una entrada en ese objeto.

5.3 Agregar o actualizar imágenes

Las imágenes se referencian por nombre de archivo desde `indice_imagenes.csv`. Para agregar imágenes:

13. Subir los archivos de imagen al servidor, en la ruta que corresponda.
14. Agregar las filas correspondientes en `indice_imagenes.csv` con el nombre correcto del archivo (columna "archivo_descargado"), la técnica asociada (columna "Técnica") y status "ok".
15. Subir el CSV actualizado al servidor.

La función `imgPath()` en `main.js` construye la URL completa de cada imagen a partir del nombre de archivo. Si las imágenes están en una subcarpeta o en un servidor diferente, hay que actualizar esa función:

```
// Funcion actual en main.js (linea ~397):
function imgPath(filename) {
  return 'imagenes/' + filename;
```



```

}

// Para imagenes en otro servidor:
function imgPath(filename) {
  return 'https://servidor.ejemplo.com/fotos/' + filename;
}

```

5.4 Modificar textos del reporte

Los textos editoriales (introducciones, callouts, descripción de secciones) están en atlas_textil_v2.html directamente en el HTML. Para modificarlos, editar ese archivo y subirlo al servidor.

Las estadísticas y graficas del reporte se calculan automáticamente desde los CSV: no requieren edición manual.

6. Datos: arquitectura actual y migración a API

6.1 Arquitectura actual: CSV estáticos

Actualmente la plataforma no consume ninguna API ni base de datos. Los datos son archivos CSV estáticos almacenados junto con el HTML en el servidor. El navegador los descarga completos al cargar la página.

Esta arquitectura tiene ventajas claras: sin servidor de aplicaciones, sin mantenimiento de base de datos, sin autenticación, sin costos de infraestructura variables. Es ideal para la fase actual del proyecto.

La limitación principal es que para actualizar los datos hay que reemplazar los archivos en el servidor manualmente.

6.2 Como migrar a una API en el futuro

Si en el futuro se desea conectar la plataforma a una API REST (por ejemplo, para actualizaciones automáticas desde KoboToolbox, o para conectar una base de datos), el cambio se concentra en un único punto: la función loadCSVs() en main.js.

Actualmente, esa función hace:

```

// Arquitectura actual (main.js, ~línea 327):
async function loadCSVs() {
  const [techRes, recRes, imgRes] = await Promise.all([
    fetch('data_by_technique_id.csv'),
    fetch('data_by_record_id.csv'),
    fetch('indice_imagenes.csv'),
  ]);
  const techRows = parseCSV(await techRes.text());
  const recordRows = parseCSV(await recRes.text());
  const imgRows = parseCSV(await imgRes.text());
  ATLAS = buildAtlasFromCSV(techRows, recordRows, imgRows);
  // ... resto de la inicializacion
}

```

Para conectar una API REST que devuelva JSON, se reemplazaría únicamente el bloque de Fetch y paseo. La función `buildAtlasFromCSV` y todo lo demás permanecen igual, siempre que el JSON devuelto tenga los mismos nombres de campo que los CSV actuales:

```
// Version con API REST hipotetica:
async function loadCSVs() {
  const [tecnicas, registros, imagenes] = await Promise.all([
    fetch('https://api.ejemplo.com/tecnicas').then(r => r.json()),
    fetch('https://api.ejemplo.com/registros').then(r => r.json()),
    fetch('https://api.ejemplo.com/imagenes').then(r => r.json()),
  ]);
  // buildAtlasFromCSV espera arrays de objetos con los mismos
  // nombres de campo que las columnas del CSV.
  ATLAS = buildAtlasFromCSV(tecnicas, registros, imagenes);
  // ... resto igual
}
```

Si la API devuelve un formato diferente al de los CSV (nombres de campo distintos, estructura anidada, paginación), habría que agregar una función de transformación entre la respuesta de la API y el formato que `buildAtlasFromCSV` espera. Esa función de transformación sería el único código nuevo necesario.

El diseño actual con `buildAtlasFromCSV` como capa de abstracción de datos fue pensado precisamente para facilitar esta migración: el resto de la aplicación (mapa, catalogo, red, árbol) no sabe si los datos vienen de CSV o de una API. Solo conocen el objeto ATLAS.

6.3 Consideraciones para una API

Si se decide implementar una API en el futuro, hay algunos aspectos a considerar:

- CORS: la API debe incluir los headers de CORS necesarios para permitir que el navegador consulte desde el dominio de UNESCO México.
- Autenticación: si la API requiere un token, este no puede estar en el código JavaScript del cliente (es publico). Se recomienda una API publica de solo lectura o un proxy del lado del servidor.
- Caching: la plataforma actualmente descarga los CSV completos en cada carga. Una API con soporte de cache HTTP (ETag, Cache-Control) mejoraría el rendimiento para datos que cambian poco.
- Paginación: `buildAtlasFromCSV` espera todos los datos de una vez. Si la API página los resultados, habría que agregar lógica para acumular todas las páginas antes de construir ATLAS.

7. Mantenimiento y diagnostico

7.1 Verificar que la plataforma carga correctamente

Al abrir la plataforma, si los CSV no se encuentran o no son accesibles, aparece un banner de error rojo en la pantalla de carga con un mensaje descriptivo. Las causas más comunes son:

- Los archivos CSV no están en la misma carpeta que `index.html`.
- Los nombres de los archivos no coinciden exactamente (mayúsculas, underscores).
- El servidor no permite acceder a los archivos CSV (problema de permisos o MIME type).

Para diagnosticar, abrir las herramientas de desarrollador del navegador (F12) y revisar la pestaña "Red" (Network). Si aparecen errores 404 en las peticiones a los CSV, es un problema de ruta. Si aparecen errores 403, es un problema de permisos en el servidor.

7.2 El mapa no muestra los estados de México

El mapa carga el GeoJSON de estados de México desde un repositorio público de GitHub (github.com/angelnmara/geojson). Si hay un problema de red o ese repositorio no está disponible, el mapa aparecerá vacío. Esto no afecta el catálogo ni las fichas.

Solución de largo plazo: descargar el archivo `mexicoHigh.json` y alojarlo en el mismo servidor que el atlas, actualizando la URL en la función `initMap()` de `main.js`.

7.3 Actualización de librerías

Las librerías externas (Leaflet, Chart.js, Google Fonts) se cargan desde CDN. En caso de necesitar actualizar a una versión específica, las URLs están en el encabezado de `index.html` y `atlas_textil_v2.html`:

```
<!-- En index.html -->
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>
```

Se recomienda no actualizar las versiones de las librerías sin verificar primero que la plataforma sigue funcionando correctamente en un entorno de prueba.

8. Formularios de contribución (KoboToolbox)

La sección "Contribuir" de la plataforma integra tres formularios de KoboToolbox. Las URLs de cada formulario están definidas en el bloque de scripts al final de `index.html`:

```
// En index.html, al final del body:
const KOBO_URLS = {
  nueva: 'https://ee.kobotoolbox.org/x/GTRuQbob',
  cambio: 'https://ee.kobotoolbox.org/x/q1s1Q5ht',
  comentario: 'https://ee.kobotoolbox.org/x/8V6qx7Dj',
};
```

Si las URLs de los formularios cambian (por ejemplo, si se crea una nueva versión del formulario en KoboToolbox), basta con actualizar esos tres valores en `index.html` y subir el archivo al servidor.

Los datos recopilados por los formularios se almacenan en la cuenta de KoboToolbox del proyecto y son independientes de la plataforma web. El proceso de incorporar nuevas contribuciones al atlas es manual: se debe revisar la cuenta de KoboToolbox, procesar los datos y actualizar los CSV.

9. Sistema de diseño

La identidad visual de la plataforma está definida en `styles.css` con variables CSS. Los colores principales son:

Variable	Valor y uso
--magenta (#B50552)	Color primario. Encabezado, acentos, botones principales, categorías "Técnicas decorativas".
--azul-mar (#035A79)	Color secundario. Links, elementos interactivos, categoría "Tejido".
--naranja (#FB4801)	Color de acento terciario. Alertas, destacados.
--verde (#05B794)	Categoría "Tenidos".
--ocre (#FFA329)	Categoría "Acabados". Números destacados en el leader.
--cian (#00B2C0)	Subcategoría "Tejidos".
--rosa (#DE599B)	Subcategoría "Aplicaciones". Género femenino en graficas.
--negro (#1A1018)	Fondo del encabezado y barras de navegación.
--va-page (#FFF8F5)	Fondo general de la plataforma.

Las tipografías se cargan desde Google Font: Francés (títulos y encabezados), DM Sans (cuerpo de texto) y Pase Mono (datos numéricos y etiquetas técnicas).